

From Job Description to Deployed Fix

A Post-Mortem

How one gap in a job description became a benchmarked C#/.NET voice agent, and what it found in a live phone line

September 23 to 24, 2026

Ariel Agor, building with Claude Code

A job posting for a Senior AI Voice Engineer asked for real-time voice-to-voice agents, telephony, speech recognition and synthesis, voice activity detection, low-latency streaming, and professional C#/.NET. I had built and run voice agents on the phone network since June 2026, all on Node.js. Rather than write around the C# gap, I built the thing in C#. This is the record of how that went: what was built, what it measured, what it broke, and what it fixed in production.

1. The decision

Remember that you're my co pilot so if you can code it, so can I.

The first read of the job description flagged three hard misses: five years of professional software engineering, a computer-science degree, and professional C#/.NET. Two of those are history and cannot be built. The third is a stack, and a stack can be. The decision was to port the call loop of my live phone line to ASP.NET Core 8, test it properly, and let the port earn the claim.

A missing *stack* is work to do. A missing *history* (years, degree, titles) is stated once and never inflated. The two are different kinds of gap and get different treatment.

2. Timeline

When (PT)	What happened
Sep 23, 18:29	First commit: G.711 codec, adaptive VAD, Twilio signature validation checked against Twilio's official library. 22 tests.
Sep 24, 12:57	The call bridge: event loop, tools, memory, barge-in, metrics, outbound calls. 89 tests.
Sep 24, 13:03	Python call simulator playing Twilio's side in real time. It found two bugs on its first run.
Sep 24, 13:17	First live calls against xAI's realtime model. Five more defects. Per-turn latency timeline added.
Sep 24, 13:32	Adapters for OpenAI Realtime, OpenAI GPT-Live and Gemini Live.
Sep 24, 13:40	Four-model benchmark: eight live calls.
Sep 24, 14:06	Fixes for three production bugs deployed to the live line as a no-traffic revision, verified with a call, then promoted.

3. What was built

- **voice-agent-dotnet:** an ASP.NET Core 8 service bridging Twilio Media Streams (8 kHz mu-law over WebSocket) to realtime speech-to-speech models. A single-consumer `Channel<T>` event loop per call; codec, stateful resampling and VAD written from scratch; local barge-in; tool calling (RAG, availability, booking, account verification, promise-to-pay); caller memory; a booking-integrity guard; outbound calls with answering-machine detection; latency metrics. At benchmark time: 3,222 lines of C#, 116 xUnit tests, Docker, CI (137 tests after review; section 9).
- **Four providers behind one loop:** xAI and OpenAI Realtime speak one event protocol; Gemini Live and OpenAI GPT-Live each needed a translating adapter, written from their docs and from raw traces of what their APIs actually sent.
- **A measurement harness** in Python: TTS-generated callers sent as real 8 kHz mu-law at real-time pace, marks echoed only after playback, full-call recordings, and a benchmark summarizer.

4. What went right

Independent checks, not self-agreement. The Twilio signature code was tested against vectors produced by Twilio’s own Node library, not against itself. Two tests were mutation-checked: disabling local barge-in and disabling the booking flush each made the right tests fail.

Measuring where the caller is. Latency was taken at the simulated caller, from the end of the caller’s speech (trailing silence removed) to the first audible agent audio. That definition caught two measurement errors described below.

Deploy dark, verify, then shift. The production fix went out as a Cloud Run revision with zero traffic, took a scripted call over its real WebSocket endpoint, emailed its call report, and only then received traffic. Rollback is one command.

The port as a test of the original. Porting forced every behaviour of the Node bridge to be restated and tested. Three of the defects it surfaced were also in the live line, and its logs confirmed them.

5. What went wrong

These are my mistakes and the process’s, stated plainly.

The tracer was the latency. To debug an under-documented API I added a trace of every provider event. The first version appended to a file synchronously, about 38 ms per event on Windows, at 50 audio frames a second. The call loop fell 3.5 seconds behind and OpenAI’s greeting went out late. Three benchmark runs were contaminated and were discarded. The trace now goes through a background queue.

Instrumentation on the hot path is part of the system under test. If the observer can move the number, it will.

Counting packets, not sound. GPT-Live is full-duplex and streams audio continuously, silence included. Measuring “first audio packet after the caller stops” reported about 300 ms. That was a fake result, and it also made every caller utterance look like a barge-in. Latency, barge-in and reply boundaries now count only audible audio.

The simulator interrupted the agent. The simulated caller waited 1.2 s of silence before speaking again. After a filler (“let me check that”), a tool round trip plus the model’s second pass takes longer than that, so the simulated caller talked over the real answer. The wait is now 2.5 s, and the dead-air gap it revealed is a real product finding.

Wrong endpoint for GPT-Live. `gpt-live-1` exists on the API key but is not a realtime-API model; it runs on a separate Live Sessions API that delegates tools to a backend model. The first attempt failed with “not supported in realtime mode”, and the adapter was built for the right API afterwards.

Deleting through a junction. To compare old and new line code side by side, I made a scratch git worktree whose `node_modules` was a Windows junction to the live repo’s. Removing the worktree with `git worktree remove --force` deleted through the junction and emptied the live repo’s `node_modules`. The laptop fallback server then failed on restart, and one run of a scheduled email agent crashed before writing a log line while Task Scheduler still reported success. Cloud Run, which takes all live traffic, was never affected. Restored with `npm ci`, both consumers re-verified, and the lesson filed so it cannot recur.

A green exit code from a launcher proves the launcher ran. It does not prove the program did. Verify the side effect.

Delegated research that did not return. Two research subagents stopped on a usage limit before reporting. The evidence was gathered by hand instead, which cost time but not accuracy.

6. Defects found by testing against live models

#	Defect	Also in production?
1	Force-reply timer answered turns already answered: xAI creates the reply about 300 ms <i>before</i> it commits the turn	Yes. 97 forced replies across ~50 answered calls (mostly test calls); 79 were followed by caller speech within 1.5 s. Fixed and deployed.
2	“Yes, that’s all correct” matched the goodbye pattern and would hang up a turn early	Yes , found in its code. Fixed and deployed.
3	xAI re-sends one utterance’s transcript as it firms up; the goodbye fired three times	Partly: transcripts were collapsed, the trigger was not. Fixed.
4	A streaming partial ending “Yes, that’s all” read as a goodbye on GPT-Live	Fixed in both.
5	Redundant response.cancel after the provider’s own interrupt failed “no active response”	Yes. 167 failures in the logs. Fixed and deployed.
6	Disarming the idle follow-up reset the tuned end-of-turn window	Port only. Fixed.
7	Silence counted as speech on a full-duplex model	Port only. Fixed.
8	Synchronous tracing delayed the call loop	Port only. Fixed.

7. The benchmark

The same five-turn service call, twice per model, through the same bridge: a price question, availability, name and phone number, a confirmation, a goodbye.

Model	Median reply	p90	Worst	Read back first
xAI grok-voice-think-fast-2.0	1.36 s	1.65 s	4.59 s	2 / 2
OpenAI gpt-live-1 (+ gpt-5.6-luna)	1.60 s	1.83 s	2.79 s	2 / 2
OpenAI gpt-realtime-2.1	1.69 s	1.95 s	2.14 s	0 / 2
Google gemini-3.8-live	1.88 s	2.13 s	2.27 s	1 / 2

Every call booked the appointment and ended itself on the caller’s goodbye, with zero model errors. OpenAI Realtime booked before confirming the caller’s details in both runs, which is disqualifying for a line that handles payments. xAI was fastest and is the only one available in the line’s cloned voice, so the live line stays on it. Ten turns per model ranks them on this task; it does not generalise beyond it.

8. The production change

The live line, +1 (775) 252-8333, is my digital twin on Google Cloud Run. It received the three production fixes, moved into a tested module with `node:test` coverage, plus a corrected ground-truth file. Before the change it told a caller asking about C# that there was no mention of it in my work, which was true until this week. After the change it says the C# work is recent and names the project.

Verification before cutover: a scripted recruiter call against the no-traffic revision over its real `wss://` endpoint, with the model and cloned voice live. The call held open through “that’s all correct” and ended on the real goodbye; the report arrived by email 31 seconds after hangup. Then 100% of traffic moved to the new revision, with the previous revision one command away.

9. After review

Two independent reviewers read the finished application cold.

An adversarial hiring manager, in the persona of the employer, advanced it to a phone screen with reservations, ran the test suite itself, and named what would move it further. Three of those items were buildable the same day and now exist, enforced in code rather than in the prompt: verbatim recorded disclosures that the model cannot paraphrase, played by the bridge while the model waits; an outbound call policy (at most 7 attempts per account in 7 days, a 7-day quiet period after a conversation, calls only between 8 a.m. and 9 p.m. local time); and a read-back gate inside the booking tool. That added 21 tests, for 137.

A truth auditor checked every claim against primary sources, not against the evidence pack, and returned “not safe to send” on the first draft. Its corrections, all applied: the 97 forced replies came from about 50 answered calls, most of them my own tests, not “about 70 real calls”; a 27 to 29 ms barge-in figure was an artifact of the test tool (a real call cannot beat about 40 ms); “fourteen failed calls” overstated what Twilio had recorded; a lost-booking story belonged to a different runtime; and a portfolio-wide commit count had included bots and a copied project’s history.

A reviewer in the reader's persona finds what a coverage check misses. An auditor with the primary sources finds what the author already believes.

10. Lessons

1. Measure what the caller hears.

Not packets, not server events: audible audio at the caller's side of the socket.

2. Test against the real provider, not its documentation.

Event ordering (reply before commit), repeated transcripts, and continuous silence were all undocumented and all mattered.

3. A port is a test suite for the original.

Restating behaviour in another language surfaced bugs that months of production had hidden.

4. The observer must not move the number.

Tracing and logging go off the hot path.

5. Deploy dark, verify on the real path, then shift traffic.

And keep the rollback to one command.

6. After touching shared state, verify every consumer.

Especially the ones whose success signal is a launcher's exit code.

11. Next

- A larger benchmark (more scripts, accents, line noise) and an end-of-turn window experiment on xAI.
- The 31-tap resampling filter from the port, into the Node line.
- A Twilio alert poll for media-path failures, which a green `/health` cannot see.
- Retire the laptop fallback now that Cloud Run has carried traffic cleanly.

*Built by directing Claude Code. The decisions, the tests, the measurements,
and the mistakes are mine.*